

Challenge Write-Up Template

Challenge Information

Student Name	Keyzia Joy Pascua
Section / Class	BSIT 3 - Networking
Challenge Name	MyGit
Category	General Skills
Points / Difficulty	Easy
Date Solved	August 11, 2026
Flag Format Given	picoCTF{...}

1. Challenge Description

MyGit is a challenge about Git commit authorship rather than about the file contents themselves. You're given a repository containing flag.txt, and the server only rewards you if flag.txt is pushed in a commit whose author identity exactly matches root:root@picoctf - meaning the real task is to reconfigure your local Git identity to impersonate that user before committing and pushing, not just to edit the file.

MY GIT

General Skills Easy

by Darkraicg492 · picoCTF 2026

I have built my own Git server with my own rules!

You can clone the challenge repo using the command below.

```
git clone ssh://git@foggy-cliff.picoctf.net:50144/git/challenge.git
```

Here's the password: **d9df7038**

Check the README to get your flag!

INSTANCE ⓘ

Expires in 5:23 Restart Instance

HINTS ⓘ

1 How do you specify your Git username and email?

FLAG ⓘ

Submit

2. Initial Analysis / Reconnaissance

Always read the README (or any provided instructions) in a Git-based challenge before touching files - it usually spells out the actual win condition. Here it explicitly states that only a commit pushed with the author root:root@picoctf will be accepted, which reframes the challenge from 'edit a file' to 'control who Git says made the edit.' That's a strong signal this challenge is really about git config user.name / user.email rather than clever file content.

3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
git clone / cd / ls -la / cat README.md	Get into the already-cloned repo and read the rules for the challenge	README.md states only flag.txt pushed by a commit authored as root:root@picoctf will be scored
git config user.name / user.email	Set the local commit identity to match the required author root / root@picoctf	git config user.name -> root, git config user.email -> root@picoctf
echo "test" > flag.txt / git add / git commit -m "push flag"	Modify flag.txt and commit the change under the root identity	Commit 1f73b73 created with the root:root@picoctf author
git remote -v / git remote set-url origin ...	Diagnose and fix a stale remote URL pointing at the wrong port	origin updated from port 61685 to the correct port 49678
git push origin master	Push the commit to the challenge server for grading	remote: Congratulations! You have successfully impersonated the root user + the flag

4. Solution Walkthrough

Step 1

Move into the repository and get oriented before changing anything: cd into the challenge folder, run ls -la to see what's there, and cat README.md to read the rules. The README makes the win condition explicit - only flag.txt pushed by a commit authored as root:root@picoctf will be updated with the flag - so the goal is to control the commit author identity, not just the file contents.

```
kyxxajoy@PASCUA:~$ git clone ssh://git@foggy-cliff.picoctf.net:49678/git/challenge.git
fatal: destination path 'challenge' already exists and is not an empty directory.
kyxxajoy@PASCUA:~$ cd challenge
kyxxajoy@PASCUA:~/challenge$ ls -la
total 20
drwxr-xr-x  3 kyxxajoy kyxxajoy 4096 Aug 11 19:00 .
drwxr-xr-x 10 kyxxajoy kyxxajoy 4096 Aug 15 15:16 ..
drwxr-xr-x  7 kyxxajoy kyxxajoy 4096 Aug 11 19:00 .git
-rw-r--r--  1 kyxxajoy kyxxajoy  155 Aug 11 18:58 README.md
-rw-r--r--  1 kyxxajoy kyxxajoy   16 Aug 11 19:00 flag.txt
kyxxajoy@PASCUA:~/challenge$ cat README.md
# MyGit

### If you want the flag, make sure to push the flag!

Only flag.txt pushed by ``root:root@picoctf`` will be updated with the flag.

GOOD LUCK!
```

Step 2

Git records an author name and email with every commit, and that identity is normally set globally or per-repo with git config. Since the challenge server checks for root:root@picocft specifically, set both values to match exactly: git config user.name "root" and git config user.email "root@picocft". Double-check with git config user.name and git config user.email before committing, since a mismatched identity (even a typo) means the push won't count.

```
pushing my flag
kyxxajoy@PASCUA:~/challenge$ git config user.email "root@picocft"
kyxxajoy@PASCUA:~/challenge$ git config user.name
git config user.email
root
root@picocft
kyxxajoy@PASCUA:~/challenge$ git status
On branch master
```

Step 3

With the identity set correctly, make a change to flag.txt so there's something new to commit - echo "test" > flag.txt works fine, since the content itself isn't what's being checked. git status confirms the file shows as modified, then git add flag.txt stages it and git commit -m "push flag" creates the commit. Because the identity was already configured, this commit is now authored as root:root@picocft.

```
nothing to commit, working tree clean
kyxxajoy@PASCUA:~/challenge$ echo "test" > flag.txt
kyxxajoy@PASCUA:~/challenge$ git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   flag.txt

no changes added to commit (use "git add" and/or "git commit -a")
kyxxajoy@PASCUA:~/challenge$ git add flag.txt
kyxxajoy@PASCUA:~/challenge$ git commit -m "push flag"
```

Additional steps (add as needed)

Before the commit can count, it has to reach the server - and a straightforward git push can fail here for reasons unrelated to the challenge logic itself. In this case, git push failed with 'Connection refused' because git remote -v showed the origin URL still pointing at an old, no-longer-valid port from a previous session (61685). Running git remote set-url origin with the correct host and port (49678, as given in the original git clone URL) fixes this. After confirming the new URL with git remote -v again, git push origin master succeeds: it connects, authenticates, and the server's own hook responds with 'Author matched and flag.txt found in commit...' followed by the flag - confirming the impersonated commit identity was exactly what unlocked it.

```
and the repository exists.
kyxxajoy@PASCUA:~/challenge$ git remote -v
origin  ssh://git@foggy-cliff.picoctf.net:61685/git/challenge.git (fetch)
origin  ssh://git@foggy-cliff.picoctf.net:61685/git/challenge.git (push)
kyxxajoy@PASCUA:~/challenge$ git remote set-url origin ssh://git@foggy-cliff
.picoctf.net:49678/git/challenge.git
kyxxajoy@PASCUA:~/challenge$ git remote -v
origin  ssh://git@foggy-cliff.picoctf.net:49678/git/challenge.git (fetch)
origin  ssh://git@foggy-cliff.picoctf.net:49678/git/challenge.git (push)
origin  ssh://git@foggy-cliff.picoctf.net:49678/git/challenge.git (push)
kyxxajoy@PASCUA:~/challenge$ git push origin master
The authenticity of host '[foggy-cliff.picoctf.net]:49678 ([3.15.249.208]):49
remote: Author matched and flag.txt found in commit...
remote: Congratulations! You have successfully impersonated the root user
remote: Here's your flag: picoCTF{1mp3rs0n4t4_g17_345y_e522152d}
```

5. Flag

picoCTF{1mp3rs0n4t4_g17_345y_e522152d}

6. Key Concept / What I Learned

The key concept is that Git commit authorship (user.name / user.email) is just local metadata you control, not a verified identity - anyone can set it to anything. This challenge is a hands-on demonstration of why server-side checks that trust the client-supplied Git author field are risky: a hook that grants access or rewards based on 'who committed this' can be trivially satisfied by anyone who knows the expected name and email, without any real authentication of who they are.

7. References

Git documentation for git config, user.name, and user.email - used to confirm the exact commands for setting and checking local commit identity, and for git remote set-url to correct a stale remote.
